RescueOps

Evolve. Adapt. Conquer. | AI agent competition | 5 rounds, 100 points

1. Problem

A rescue control room receives one or more rescue missions. Your job is to build an **AI agent** that reads the scenario and returns a **safe and smart decision**: which mission to send help to (or whether to wait, reject or resupply). You write the strategy in **agent.py** using **LangGraph + LangChain tools**. Round 5 also uses an LLM through **Ollama**.

One agent handles every round: the evaluator calls your `choose_rescue(case)` with cases from all rounds, and each case carries a `round` field.

2. The five rounds

Round	Points	Scenario	Your agent must handle
1. Basic Rescue	10	One mission, clear weather, no blocked road	Is it feasible (fuel and deadline)? Dispatch or not
2. Blocked Routes	15	1-2 missions; `route_blocked` and `alternate_distance` appear	Never use a blocked road; consider the alternate route (it may not exist)
3. Limited Resources	20	2-3 missions; fuel / kits / personnel limited; `resupply_possible` may be true	Resource checks; what to do when nothing fits
4. Critical Priorities	25	3-5 missions with tight deadlines	Choose well between competing missions (severity vs feasibility)
5. Adaptive RescueOps	30	Everything combined. Weather can change; later stages bring a free-text `update_report`	Read the report, update the scenario, decide again. **Needs an Ollama call**

Each round includes everything from the earlier ones, so Round 1 logic must still work in Round 5.

3. The scenario (input)

```
case_id, round, stage, weather ("clear" | "rain" | "storm"), resupply_possible (bool),
resources: { fuel, kits, personnel },
missions: [ { id, severity 1-5, distance, deadline, kits, personnel,
              route_blocked, alternate_distance | null } ],
update_report (Round 5, stage > 0)
```

Rules of the world

- One mission is chosen per decision.
- **Fuel needed** = ceil(route km / 10).
- **Travel time** = ceil(km x weather factor), with clear = 1.0, rain = 1.25, storm = 1.5. It must be <= the mission `deadline`.
- A **blocked** direct route cannot be used. Use the alternate route (if `alternate_distance` is not null) by returning `uses_alternate_route: true` (or action `reroute`).

- Kits and personnel needed come from the mission; your resources must cover fuel, kits and personnel.
- Round 5 calls your agent once per stage; each call must reflect the **current** state.

4. The decision (output)

```
{"action": "dispatch", "priority": 4, "reason": "Mission is urgent and feasible",
 "mission_id": "M1", "uses_alternate_route": false,
 "fuel_needed": 2, "kits_needed": 1, "personnel_needed": 1}
```

Field	Rule
action	**Required.** One of: dispatch, wait, reroute, reject, resupply
priority	**Required.** Integer 1-5
reason	**Required.** Non-empty text
mission_id	Required for dispatch / reroute when the case has more than one mission
uses_alternate_route	Optional bool. Must be true (or use action reroute) to take the alternate route
fuel_needed, kits_needed, personnel_needed	Optional non-negative ints. They are **checked, not trusted**

5. How you are scored

- **Per case: 70% decision output + 20% tool usage + 10% speed.**
- **Tool usage:** the evaluator sees which of your LangChain tools are actually *invoked*: `check_route`, `check_resources`, `check_deadline`, plus `parse_update_report` when an `update_report` exists. Tool names must match exactly and be called with `.invoke({...})`.
- **Speed:** full marks up to 0.3 s per case in Rounds 1-4, so do **not** call the LLM there. Round 5 gets a bigger LLM time budget.
- **Safe, feasible, high-severity** choices earn the most. Doing nothing when a feasible rescue exists earns little. Correctly refusing / waiting / resupplying when nothing is feasible earns well.
- **Unsafe or malformed output zeroes the whole case**: dispatch through a blocked route, not enough resources, missed deadline, unknown mission, wrong format, crash or timeout. Unsafe dispatches also count as safety violations; **fewer violations break ties**.
- Rounds are worth 10 / 15 / 20 / 25 / 30 = **100 points**. Hidden test cases differ from the samples, so hard-coded answers will fail. Pretty prose is worth nothing.

6. Round 5 and Ollama (mandatory for Round 5)

- In later stages the scenario fields are **stale**. The real changes arrive only as free text in `case["update_report"]`, for example: *"Weather is now storm. Route to M2 is now blocked. Fuel is now 4."* You are scored against the updated truth.
- **A Round 5 case scores 0 unless your agent makes at least one successful Ollama call** (HTTP 200 to ollama.com) while handling that case, using the Python `requests` library.
- Enter your own Ollama API key and model name in the submit form for Round 5. They are used only for that run and are not stored. **Never put a key in your code**; use environment variables `OLLAMA_API_KEY` and `OLLAMA_MODEL` locally. Use a limited or throwaway key and revoke it afterwards.

- Use the LLM to read the report, then **verify its answer yourself** (blocked route? resources? deadline?). An invalid LLM answer scores 0 for that stage.
- Rounds 1-4 run **without** any key, so your agent must work offline there.

7. Submission (round by round)

- Submit **one round at a time, in order**: Round 1 first, then Round 2, then 3, 4 and 5. A round unlocks after the previous one is submitted.
- You can resubmit any round before the deadline. The organizers keep your **best result for each round**; your total is the sum of those bests.
- Upload `agent.py` alone, or a ZIP with `agent.py` at the top level plus any helper `.py` / `.json` files you import. ZIP must be under 1 MB.
- **Never include:** API keys, `.env`, tokens, `.venv`, `__pycache__`, or anything copied from another team. Files with keys are rejected.
- Team name: 2-40 letters / numbers / spaces / - / _.
- **Scores are not shown to participants**; the organizers will announce the results.
- Submission page: **http://10.50.20.214:5000** (same network as the organizer).

8. Starter kit and local testing

The participant pack contains `agent.py` (starter), `app.py` (local checker), `examples/sample_cases.json` and a submission checklist. Sample cases are named by round: r1_p1, r1_p2, ... r5_p1, r5_p2.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1          (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt

python app.py                          # run all sample cases
python app.py --case r2_p1 --show      # one case, with scenario and decision
python app.py --test-api               # check your Ollama setup (Round 5)
```

The local checker verifies format and hard safety rules only; it does not give your competition score.

9. Suggested plan

- Write the three tools as LangChain `@tool`s and a small LangGraph (read, analyze, decide).
- Get Round 1 passing first, then add blocked routes (R2), resources and many missions (R3-R4).
- Last: Round 5. Add `parse_update_report`, set your Ollama key, run `python app.py --test-api`, then test r5_p1.